



上海财经大学
SHANGHAI UNIVERSITY OF FINANCE AND ECONOMICS

《数据科学与数据分析基础》课程

第三讲 数据采集

授课教师：袁浩禹

2026 年 03 月

上海财经大学信息管理与工程学院





杂论1

- 课程分享

➤ 课程分享[周日中午**12**点前报名（邮件/企微） 周一中午**12**点前PPT发送 **5-10**分钟]

✓ 爬虫的“盗亦有道”

- Robots.txt 真的能“保护”网站数据不被爬取吗？
- 国内外对于爬虫态度的对比？
- 如果你是某平台的开发者，你会如何保护你的数据不被爬取？及相关案例



杂论2

• 作业零 选题构思

✓ 已发布

编辑

⋮

请结合个人的兴趣爱好，提交一个数据分析选题（200字以内）。

选题重点考虑以下方面：

- 重要
- 有趣
- 数据可获得

每周课前老师会反馈一个10分制的分数，该分数仅做选题参考，不计课程分数。



课程回顾

- Python语言
- Python的安装与开发环境配置
- Python基本语法
- HTML简介
- 互联网络协议
- 网络爬虫的“盗亦有道”



本讲纲要

- 网络爬虫的“盗亦有道”
- 网站信息及查看工具
- Requests库入门
- 正则表达式



网站小工具

- 网站爬虫协议： robots.txt
- 网站的结构： Sitemap
- 网站使用的技术栈： wad -u url命令
- 使用whois命令
- 浏览器的开发者工具（chrome）



- pages.ctrip.com/public/sitemap/sitemap.htm

YouTube

地图

上财认证

Google 学术搜索

Tsinghua Universit...

GitHub - REMitch...

Google 翻译

清华大学图书馆

清华大学图书馆

GitHub - yeungsk/...

北京酒店

上海酒店

广州酒店

深圳酒店

重庆酒店

三亚酒店

杭州酒店

厦门酒店

澳门酒店

南京酒店

苏州酒店

长沙酒店

太原酒店

西安酒店

成都酒店

福州酒店

大连酒店

南宁酒店

天津酒店

郑州酒店

昆明酒店

沈阳酒店

合肥酒店

哈尔滨酒店

青岛酒店

石家庄酒店

桂林酒店

黄山酒店

贵阳酒店

东莞酒店

珠海酒店

西宁酒店

兰州酒店

泰安酒店

长春酒店

乌鲁木齐酒店

武汉酒店

南昌酒店

温州酒店

张家界酒店

丽江酒店

香港酒店

九寨沟酒店

海口酒店

银川酒店

千岛湖酒店

济南酒店

呼和浩特酒店

更多城市酒店预订>

更多品牌酒店预订>

更多地标附近酒店预订>

城市大全>

酒店团购

北京酒店团购

长沙酒店团购

成都酒店团购

承德酒店团购

大理酒店团购

大连酒店团购

凤凰酒店团购

佛山酒店团购

广州酒店团购

桂林酒店团购

杭州酒店团购

黄山酒店团购

昆明酒店团购

拉萨酒店团购

丽江酒店团购

溧阳酒店团购

临安酒店团购

庐山酒店团购

洛阳酒店团购

南昌酒店团购

南京酒店团购

青岛酒店团购

三亚酒店团购

上海酒店团购

绍兴酒店团购

深圳酒店团购

沈阳酒店团购

苏州酒店团购

泰安酒店团购

天津酒店团购

芜湖酒店团购

武汉酒店团购

婺源酒店团购

西安酒店团购

厦门酒店团购

烟台酒店团购

宜昌酒店团购

重庆酒店团购

峨眉山酒店团购

九寨沟酒店团购

连云港酒店团购

南戴河酒店团购

更多集团酒店预订>

海外酒店预订

新加坡酒店

普吉岛酒店

曼谷酒店

首尔酒店

东京酒店

吉隆坡酒店

巴厘岛酒店

皮皮岛酒店

芭堤雅酒店

济州岛酒店

哥打京那巴鲁酒店

胡志明市酒店

巴黎酒店

兰卡威酒店

清迈酒店

伦敦酒店

大阪酒店

台北酒店

暹粒酒店

马尼拉酒店

洛杉矶酒店

迪拜酒店

温哥华酒店

民丹岛酒店

法兰克福酒店

纽约酒店

罗马酒店

长滩岛酒店

悉尼酒店

雅加达酒店

更多海外城市酒店预订>



- 网站使用的技术栈：wad.detection

```
: # 查看网站的技术类型
# http://www.12306.cn/
# https://blog.csdn.net/weixin_45906894/article/details/109014209?spm=1001.2014.3001.5502

import wad.detection
det=wad.detection.Detector()
url=input()
print(det.detect(url))

http://www.12306.cn/
{'https://www.12306.cn/index/': [{'app': 'jQuery', 'ver': None, 'type': 'JavaScript Libraries'}, {'app': 'RequireJS', 'ver': None, 'type': 'JavaScript Frameworks'}, {'app': 'Handlebars', 'ver': None, 'type': 'JavaScript Frameworks'}]}
```

该网站使用了以下技术：

- jQuery**（JavaScript 库）：一种流行的 JavaScript 库，主要用于简化 DOM 操作、事件处理、AJAX 请求等。
- RequireJS**（JavaScript 框架）：一种 JavaScript 模块加载器，常用于管理前端代码的依赖关系。
- Handlebars**（JavaScript 框架）：一个前端模板引擎，主要用于生成 HTML 页面。



调研网站

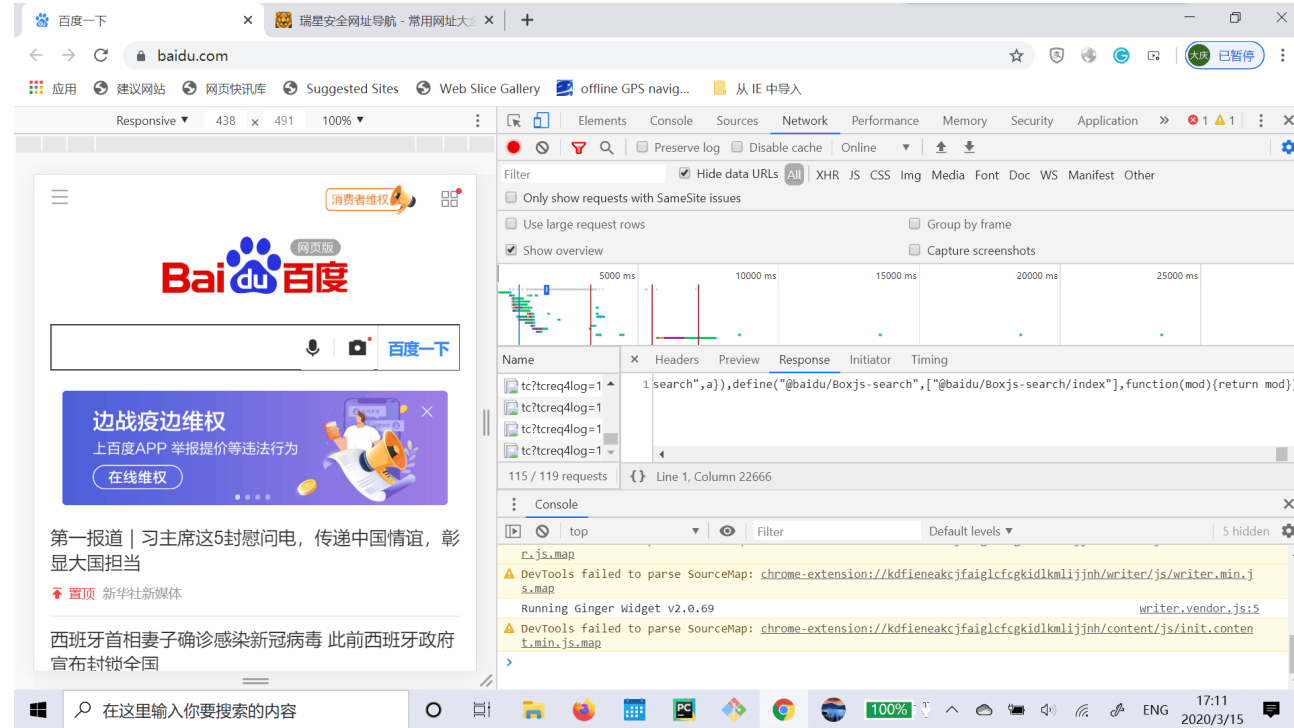
- 网站的域名信息：whois命令

```
[5]: # pip install python-whois
import whois as wh
wh.whois('yale.edu')
```

```
[5]: {'domain_name': 'YALE.EDU',
      'creation_date': datetime.datetime(1987, 3, 17, 0, 0),
      'last_modified': '05-Nov-2024',
      'expiration_date': datetime.datetime(2027, 7, 31, 0, 0)}
```



浏览器的开发者工具 (chrome)





浏览器开发者工具 (chrome)

The screenshot shows the Chrome DevTools Network tab. The top toolbar includes tabs for Elements, Console, Sources, Network, Performance, Memory, Application, Security, Lighthouse, and Recorder. The Network tab is active, displaying a list of requests on the left and a detailed view of a selected request on the right.

Network Tab List:

- newfanyi-da0cea8f7e.png
- newxueshuicon-a5314d5c83.png
- newwenku-d8c9b7b0fb.png
- newbaike-889054f349.png
- newzhidao-da1cf444b0.png
- newjiankang-f03b804b4b.png
- yingxiaoicon-612169cc36.png
- newzhibo-a6a0831ecd.png
- newyinyue-03ecd1e9b9.png
- iconfont-219cdb5f8e.woff2
- aria-3006e33cce.png
- qrcode@2x-daf987ad02.png

Selected Request Details:

- Request URL:** chrome-extension://efaidnbmninnibpcapjcgclclefindmkaj/data/js/extn-utils.html
- Request Method:** GET
- Status Code:** 200 OK
- Referrer Policy:** unsafe-url
- Response Headers:**
 - Access-Control-Allow-Origin: *
 - cache-control: no-cache
 - Content-Security-Policy: script-src 'self' https://use.typekit.net https://assets.adobedtm.com https://*.adobecontent.io https://*.adobelogin.com https://*.acrobat.com https://*.adobe.com; child-src 'self' https://*.adobecontent.io https://*.adobelogin.com https://*.acrobat.com https://*.adobe.com https://*.adobelogin.com; objec



本讲纲要

- 网站信息及查看工具
- **Requests库入门**
- 正则表达式



爬取网页初探——Requests库介绍

➤ Requests库的7个主要方法

方法	说明
<code>requests.request()</code>	构造一个请求，支撑以下各方法的基础方法
<code>requests.get()</code>	获取HTML网页的主要方法，对应于HTTP的GET
<code>requests.head()</code>	获取HTML网页头信息的方法，对应于HTTP的HEAD
<code>requests.post()</code>	向HTML网页提交POST请求的方法，对应于HTTP的POST
<code>requests.put()</code>	向HTML网页提交PUT请求的方法，对应于HTTP的PUT
<code>requests.patch()</code>	向HTML网页提交局部修改请求，对应于HTTP的PATCH
<code>requests.delete()</code>	向HTML页面提交删除请求，对应于HTTP的DELETE



Requests.get()方法

requests.get()

```
r = requests.get(url)
```

返回一个包含服务器
资源的Response对象

Response

构造一个向服务器请求
资源的Request对象

Request



Python爬虫初步：Requests库入门

➤ Requests 初探

```
import requests

r = requests.get('https://www.baidu.com/')
print(type(r))
print(r.status_code)
print(type(r.text))
print(r.text[:1000])
```

```
<class 'requests.models.Response'>
200
<class 'str'>
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equiv=content-type content=text/html; charset=utf-8><meta http-equiv=X-UA-Compatible content=IE=Edge><meta content=
=always name=referrer><link rel=stylesheet type=text/css href=https://ss1.bdstatic.com/5eN1bjq8AAUYm2zgoY3K/r/www/cache/bdorz/baidu.min.css><title>百度一下，你就知道</title></head> <body link=#0000cc> <div id=wrapper> <div id=head> <div class=head_wrapper> <div class=s_form> <div class=s_form_wra
pper> <div id=lg> <img hidefocus=true src=//www.baidu.com/img/bd_logo1.png width=270 height=129> </div> <form id=form name=f action=//www.baidu.com/s cla
ss=fm> <input type=hidden name=bdorz_come value=1> <input type=hidden name=ie value=utf-8> <input type=hidden name=f value=8> <input type=hidden name=rsv
_bp value=1> <input type=hidden name=rsv_idx value=1> <input type=hidden name=tn value=baidu><span class="bg s_ipt_wr"><input id=kwd name=wd class=s_ipt v
alue maxlength=255 autocomplete=off autofocus></span><s
```




爬取网页初探——Requests库介绍

➤ HTTP的常见状态码

HTTP常见状态码 (200、201、304、400、404、500) 等错误

一、常见的状态码

- 200 (成功) - 服务器成功返回网页
- 201 (已创建) - 请求成功并且服务器创建了新的资源。
- 304 (未修改) - 自从上次请求后，请求的网页未修改过。服务器返回此响应时，不会返回网页内容。
- 400 (错误请求) - 服务器不理解请求的语法。
- 404 (未找到) - 请求的网页不存在
- 500 (服务器内部错误) - 服务器遇到错误，无法完成请求。
- 503 (服务不可用) - 服务器目前无法使用 (由于超载或停机维护)。通常，这只是暂时状态。

二、HTTP状态码100-500详细介绍

1XX (临时响应信息提示)

这些状态代码表示临时的响应。客户端在收到常规响应之前，应准备接收一个或多个1xx。

100 (继续) - 请求者应当继续提出请求。服务器返回此代码表示已收到请求的第一部分，正在等待其余部分。

101 (切换协议) - 服务器将遵从客户的请求转换到另外一种协议

2XX (成功)

表示成功处理了请求的状态代码。

200 (成功) - 服务器已成功处理了请求。

201 (已创建) - 请求成功并且服务器创建了新的资源。

202 (已接受) - 服务器已接受请求，但尚未处理。

203 (非授权信息) - 服务器已成功处理了请求，但返回的信息可能来自另一来源。

204 (无内容) - 服务器成功处理了请求，但没有返回任何内容。

205 (重置内容) - 服务器成功处理了请求，但没有返回任何内容。与 204响应不同，此响应要求请求者重置文档视图(例如，清除表单内容以输入新内容)。

206 (部分内容) - 服务器成功处理了部分 GET 请求。

3XX - 重定向

表示要完成请求，需要进一步操作。客户端浏览器必须采取更多操作来实现请求。例如，浏览器可能不得不请求服务器上的不同的页面，或通过代理服务器重复该请求。建议在每次请求中使用重定向不要超过 5次。通常，这些状态代码用来重定向。

300 (多种选择) - 针对请求，服务器可执行多种操作。服务器可根据请求者 (user agent) 选择一项操作，或提供操作列表供请求者选择。

301 (永久移动) - 请求的网页已永久移动到新位置。服务器返回此响应 (对 GET 或 HEAD 请求的响应) 时，会自动将请求者转到新位置。

302 (临时移动) - 服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。

303 (查看其他位置) - 请求者应当对不同的位置使用单独的 GET 请求来检索响应时，服务器返回此代码。

304 (未修改) - 自从上次请求后，请求的网页未修改过。服务器返回此响应时，不会返回网页内容。

305 (使用代理) - 请求者只能使用代理访问请求的网页。如果服务器返回此响应，还表示请求者应使用代理。

307 (临时重定向) - 服务器目前从不同位置的网页响应请求，但请求者应继续使用原有位置来进行以后的请求。

4XX (请求错误)

这些状态码表示请求可能出错，客户端似乎有问题。例如，客户端请求不存在的页面，客户端未提供有效的身份验证信息。妨碍了服务器的处理。

400 (错误请求) - 服务器不理解请求的语法。

401 (未授权) - 请求要求身份验证。对于登录后请求的网页，服务器可能返回此响应。

403 (禁止) - 服务器拒绝请求。如果您在 Googlebot 尝试抓取您网站上的有效网页时看到此状态码(您可以在 Google网站管理员工具诊断下的网络抓取页面上看到此信息)，可能是您的服务器或主机拒绝了 Googlebot 访问。

404 (未找到) - 服务器找不到请求的网页。例如，对于服务器上不存在的网页经常会返回此代码。

405 (方法禁用) - 禁用请求中指定的方法。

406 (不接受) - 无法使用请求的内容特性响应请求的网页。

407 (需要代理授权) - 此状态码与 401(未授权)类似，但指定请求者应当授权使用代理。如果服务器返回此响应，还表示请求者应当使用代理。

408 (请求超时) - 服务器等候请求时发生超时。

409 (冲突) - 服务器在完成请求时发生冲突。服务器必须在响应中包含有关冲突的信息。服务器在响应与前一个请求相冲突的 PUT请求时可能会返回此代码，以及两个请求的差异列表。

410 (已删除) - 如果请求的资源已永久删除，服务器就会返回此响应。该代码与404 (未找到) 代码类似，但在资源以前存在而现在不存在的情况下，有时会用 410 来替代 404 代码。如果资源已永久移动，您应使用 301指定资源的新位置。

411 (需要有效长度) - 服务器不接受不含有效内容长度标头字段的请求。

412 (未满足前提条件) - 服务器未满足请求者在请求中设置的其中一个前提条件。

413 (请求实体过大) - 服务器无法处理请求，因为请求实体过大，超出服务器的处理能力。

414 (请求的 URI 过长) - 请求的 URI (通常为网址)过长，服务器无法处理。

415 (不支持的媒体类型) - 请求的格式不受请求页面的支持。

416 (请求范围不符合要求) - 如果页面无法提供请求的范围，则服务器会返回此状态码。

417 (未满足期望值) - 服务器未满足“期望”请求标头字段的要求。



Python爬虫初步：Requests库入门

➤ Requests 传入参数

```
data = {  
    'name': 'germey',  
    'age': 22  
}  
r = requests.get("http://httpbin.org/get", params=data)  
print(r.text)  
  
{  
  "args": {  
    "age": "22",  
    "name": "germey"  
  },  
  "headers": {  
    "Accept": "*/*",  
    "Accept-Encoding": "gzip, deflate, br, zstd",  
    "Host": "httpbin.org",  
    "User-Agent": "python-requests/2.32.3",  
    "X-Amzn-Trace-Id": "Root=1-67ce81b2-6fd0b3e368db8ff33e4ea271"  
  },  
  "origin": "103.172.183.206",  
  "url": "http://httpbin.org/get?name=germey&age=22"  
}
```



Python爬虫初步：Requests库入门

➤ Requests 传入参数

```
data = {'name': 'germey', 'age': '25'}
r = requests.post("https://httpbin.org/post", data=data)
print(r.text)

{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "age": "25",
    "name": "germey"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, br, zstd",
    "Content-Length": "18",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.3",
    "X-Amzn-Trace-Id": "Root=1-67ce8610-78ac17c26b59747b688689ae"
  },
  "json": null,
  "origin": "103.172.183.206",
  "url": "https://httpbin.org/post"
}
```



Python爬虫初步：Requests库入门

➤ Requests 传入参数

```
import requests

def get_user_posts(user_id):
    url = "https://jsonplaceholder.typicode.com/posts"
    #https://jsonplaceholder.typicode.com/posts?userId=3

    # 传递的参数
    params = {
        'userId': user_id # 通过userId参数指定用户
    }

    # 发送GET请求
    response = requests.get(url, params=params)
    print("返回的网页:", response.text)
    # 检查请求是否成功
    if response.status_code == 200:
        # 解析JSON响应
        posts = response.json()
        print(f"用户 {user_id} 的帖子: ")
        for post in posts:
            print(f"标题: {post['title']}")
            print(f"内容: {post['body']}")
            print("---")
    else:
        print(f"错误: {response.status_code} - {response.text}")

# 获取用户ID
user_id = input("请输入用户ID (1到10): ")

# 调用函数
get_user_posts(user_id)
```



Python爬虫初步：Requests库入门

➤ Requests header信息

```
headers = {'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/52.0.2743.116 Safari/537.36' }
```

```
r = requests.get("https://www.zhihu.com", headers=headers)  
print(r.text)
```

```
r = requests.get("https://www.zhihu.com")  
print(r.text)
```

```
{"error":{"message":"您当前请求存在异常，暂时限制本次访问。如有疑问，您可以通过手机摇一摇或登录后私信知乎小管家反馈。"},"code":40362}}
```



```
r = requests.get('https://github.com/favicon.ico')
print(r.text)
print(r.content)

#with open('favicon.ico', 'wb') as f:
#    f.write(r.content)
```

2026/3/16



Python爬虫初步: Requests库入门

➤ Requests 传入信息

```
files = {'file': open('favicon.ico', 'rb')}  
r = requests.post('https://httpbin.org/post', files=files)  
print(r.text)
```



```
{  
  "args": {},  
  "data": "",  
  "files": {  
    "file": "data:application/octet-stream;base64,AAABAAIAEBAAAAEIAAoBQAAJgAAACAgAAABACAABQAAE4FAAAoAAAAEAAAACAAAAABACAAAAAAAFAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAABERE3YTeXPFDg40EgAAAAAAAAAADw8PERERFLETExNpAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABQUFJYTeXT8ExMU7QAAABkAAAAAAAAAAAAAAAAABgVFRf/FR  
UX/xERE4UAAAAAAAAAAAAAAAAAAAAAAAAABEREsETExTuERERHhAQEBAAAAAAAAAAAAAAAAAAAAAAAAANExMU9RUVF/8VFRf/ERUrwAAAAAAAAAAAAAAAAABQUFJkVFRf/BgYRLA4ODlwPDw/BDw8PIgAA  
AAAAAAAAADw8PNBAQEP8VFRf/FRUX/xUVF/8UFBSPAAAAABAQEDAPDQ//AAAA+QEBAe0CAgL/AgIC9g4ODjgAAAAAAAAAAgICEACAgLrFRUX/xUVF/8VFRf/FRUX/xERES0UFBWcFBQV/wEBAfwPDx  
H7DQ0R0wAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA0NEjoTExTnFRUX/xUVF/8SEhKaExMT2RUVF/8VFRf/ExMTTwAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAERERTBUVf/8VFRf/ExMT  
2hMTFPYVFRf/FBQU8AAAAAIAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAITEXTxFRUX/xMTFPYTeXT3FRUX/xQUFOEAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAFBQU4RUVF/8TEXT3FBQU3hUVF/8TEXT5Dw8PIQAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAEBQHxMTFPgVFRf/FBQU3HERFKIVFRf/FRUX/w8PDzQAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAABAQEEAVFRf/FRUX/xERFKIODg44FRUX/xUVF/8SEhKYAAAAAAAAAwAAAAKAAAAAAAAAAAAAAAAMAAAAAQAAAASEhKYFRUX/xUVF/8ODg44AAAAABERFKQVFRf/ERESwQ  
4ODjYAAACBDQ0N3BISFNgSEhTYExMU9wAAAHQBQU3ERESwRUVF/8RERSkAAAAAAAAAAAAAAAAADExMTxhUVF/8VFRf/FRUX/xUVF/8VFRf/FRUX/xUVF/8VFRf/FRUX/xUVF/8TExPGAAAAAwAAAAA  
AAAAAAAAAAAAAAAAAMRERSiFRUX/xUVF/8VFRf/FRUX/xUVF/8VFRf/FRUX/xUVF/8RERSiAAAAAwAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABAQED4TExOXExMT2RISFPIShTyExMT2RMTE5cQEBA+AA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA  
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```



Python爬虫初步：Requests库入门

➤ Requests cookie查看与应用

```
r = requests.get('https://www.baidu.com')
print(r.cookies)
for key, value in r.cookies.items():
    print(key + '=' + value)
```

```
<RequestsCookieJar[<Cookie BDORZ=27315 for .baidu.com/>]>
BDORZ=27315
```

```
headers = {
    'Cookie': '_xsrp=xuSc5JAUL6TF8gLvnt219xCtoRgugTm3; _zap=fcc0acd2-aab8-4aa1-a6b6-3088f464444a; d_c0=ABASIC_89xmPTo4qaG10kA28BzKezYh2LxE=|1738918727; Hm'
    'Host': 'www.zhihu.com',
    'User-Agent': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_11_4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/53.0.2785.116 Safari/537.36',
}
r = requests.get('https://www.zhihu.com', headers=headers)
print(r.text)
```

```
<!doctype html>
<html lang="zh" data-hairline="true" class="itcauecng" data-theme="light"><head><meta charset="utf-8"/><title data-rh="true">首页 - 知乎</title><meta n
ame="viewport" content="width=device-width,initial-scale=1,maximum-scale=1"/><meta name="renderer" content="webkit"/><meta name="force-rendering" cont
ent="webkit"/><meta http-equiv="X-UA-Compatible" content="IE=edge,chrome=1"/><meta name="google-site-verification" content="&quot;FTeR0c8arOPKh8c5DYh_
9uu98_zJbaWw53J-Sch9MTg&quot;"/><meta name="description" property="og:description" content="知乎，中文互联网高质量的问答社区和创作者聚集的原创内容平台，于 2
```



Python爬虫初步：Requests库入门

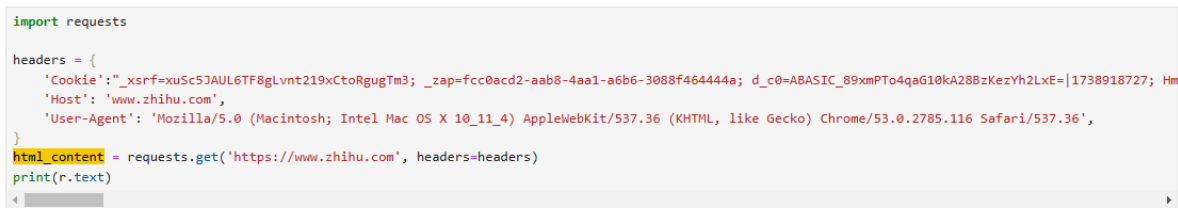
➤ Timeout的使用

```
r = requests.get('https://www.google.com', timeout=10)
print(r.status_code)
```




本讲纲要

- 网络爬虫的“盗亦有道”
- 网站信息及查看工具
- Requests库入门
- 正则表达式

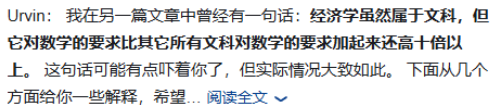
[关注](#) [推荐](#) [热榜](#) [视频](#)

一个月学python来得及吗？

三月三：你问来不来得及？这得看你打算用Python来做什么。见过通宵三天用pandas处理实验数据通过答辩，也见过计算机大四学生写个冒泡排序还要百度... [阅读全文](#)

▲ 赞同 3 ▼ 1 条评论 分享 收藏 喜欢 ...

文科生学经济学难吗？



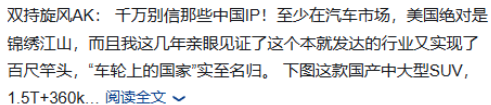
▲ 赞同 26 ▼ ● 16 条评论 ↗ 分享 ★ 收藏 ♥ 喜欢 ..

我一个在互联网大厂工作了十年的中年男人，离职后投了几十封简历却没人要，该怎么办？

当时明月在：降低期待继续投吧，你们的思想钢印太重了。最近这些年里，我对于经济的大方向的判断，说实话很少出错。我前几年意识到了这个风险，所以干脆主动出... [阅读全文](#) 

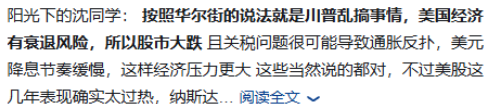
▲ 赞同 384 ▼ ● 39 条评论 ↗ 分享 ★ 收藏 ♥ 喜欢 ...

美国为什么越来越差了？



▲ 赞同 174 ▼ ● 38 条评论 ↗ 分享 ★ 收藏 ♥ 喜欢 ..

美股遭遇「黑色星期一」，三大指数集体收跌，特斯拉跌超 15%，发生了什么？有哪些信息值得关注？



▲ 赞同 1961 ▼ ● 299 条评论 ↗ 分享 ★ 收藏 ♥ 喜欢 ..



问题提出

➤ 举个栗子

- 假设你在一篇英文小说里查找hi，你可以使用”hi”。
- `'\bhi\b'`
- ‘`\b`’ 是正则表达式规定的一个特殊代码（代表元字符，**meta-character**），代表着单词的开头或结尾，也就是单词的分界处。虽然通常英文的单词是由空格，标点符号或者换行来分隔的，但是`\b`并不匹配任何一个这些单词分隔字符中，它只匹配一个位置。
 - 元字符：正则表达式中具有一些特殊含义的字符，例如重复N次前面的字符等。

推荐正则表达式学习网页：
<https://deerchao.cn/tutorials/regex/regex.htm#mission>
<https://www.runoob.com/python/python-reg-expressions.html>

举个“栗”子





正则表达式基础知识

➤ 正则表达式

描述字符串排列的一套规则，通常为了更好的“匹配”或者“寻找”符合某个规则（模式）的文本和字符串，许多程序设计语言都支持利用正则表达式进行字符串操作，又称规则表达式。

— 原子

正则表达式中最基本的组成单位，每个正则表达式至少包含一个原子。

— 元字符

正则表达式中具有特殊意义的字符，用来匹配特定类型的字符或位置

— 字符集

字符集用于匹配一组字符中的任一个

— 量词

量词用于指定前一个字符或分组出现的次数



正则表达式：原子

- 原子：每个正则表达式至少包含一个原子
 - 普通字符作为原子：包括数字、大小写字符、下划线等；

#正则表达式 原子

```
import re
```

```
content = 'Hello 123 4567 World_This is a Regex Demo, Hello, my phone number is 021-65901447 and  
print(len(content))  
result = re.match('Hello', content)  
print(result)  
print(result.span())  
print(result.group())
```

158

```
<re.Match object; span=(0, 5), match='Hello'>
```

```
(0, 5)
```

```
Hello
```



正则表达式常见函数

- Python的re库
 - **re.match(pattern, string[, flags])** # 如果字符串开头的零个或多个字符与正则表达式模式匹配，则返回相应的匹配对象。
 - **re.search(pattern, string[, flags])** # 扫描字符串以查找正则表达式模式产生匹配项的第一个位置，然后返回相应的**match**对象。
 - **re.compile(pattern[, flag])** # 将正则表达式编译为正则表达式对象
 - **re.sub(pattern, repl, string[, count])** # 返回通过用替换repl替换字符串中最左边的不重叠模式所获得的字符串。
 - **re.split(pattern, string[, maxsplit])** # 通过出现模式来拆分字符串。
 - **re.findall(pattern, string[, flags])** # 以string列表形式返回string中pattern的所有非重叠匹配项。
 - **re.finditer(pattern, string[, flags])** # 返回一个迭代器，该迭代器在string类型的RE 模式的所有非重叠匹配中产生匹配对象。
 - **re.subn(pattern, repl, string[, count])** # 执行与相同的操作sub()，但返回一个元组。



正则表达式：原子

➤ 原子：每个正则表达式至少包含一个原子

模式	含义	模式	含义	模式	含义
\w	匹配任意一个字母、数字或下划线	\A	匹配字符串开始	^	匹配字符串的开头
\W	匹配除字母、数字、下划线以外的任意一个字符	\Z	匹配字符串结束，如果存在换行，只匹配到换行前的结束字符串	\$	匹配字符串的末尾
\s	匹配任意一个空白字符	\z	匹配字符串结束	.	匹配任意字符，除了换行符
\S	匹配除空白字符以外的任意一个其他字符	\G	匹配最后匹配完成的位置	[...]	用来表示一组字符，单独列出
\d	匹配任意一个十进制数，等价于[0-9]	\n	匹配一个换行符	[^...]	匹配不在[]中的字符
\D	匹配除十进制数以外的任意一个其他字符	\t	匹配一个制表符	*	匹配0个或多个的表达式
{n}	重复n次	{n, m}	表示n到m次	+	匹配1个或多个的表达式



正则表达式基础知识：元字符

➤ 元字符

— 任意匹配元字符

- 用 ‘.’ 匹配一个除了换行符以外的任意字符；

符号	含义	符号	含义
.	匹配除了换行符以外的任意字符	{n}	前面的原子恰好出现n次
^	匹配字符串的开始位置	{n,}	前面的原子至少出现n次
\$	匹配字符串的结束位置	{n, m}	前面的原子至少出现n次，至多出现m次
*	匹配0次、1次或多次前面的原子		模式选择符，从多个模式中任意选择一个模式匹配
?	匹配0次、1次前面的原子	()	模式单元符，将一些原子组合成一个大原子使用
+	匹配1个或多次前面的原子		



正则表达式基础知识：元字符

➤ 元字符

– 边界限制元字符

- 用 ‘^’ 匹配字符串的开始，用 ‘\$’ 匹配字符串的结束；

```
content = 'Are you a writer or a doctor'
print(len(content))
result = re.findall('or', content)
print(result)
result = re.findall('^or', content)
print(result)
result = re.findall('or$', content)
print(result)
```

28
['or', 'or']
[]
['or']

符号	含义	符号	含义
.	匹配除了换行符以外的任意字符	{n}	前面的原子恰好出现n次
^	匹配字符串的开始位置	{n,}	前面的原子至少出现n次
\$	匹配字符串的结束位置	{n, m}	前面的原原子至少出现n次，至多出现m次
*	匹配0次、1次或多次前面的原子		模式选择符，从多个模式中任意选择一个模式匹配
?	匹配0次、1次前面的原子	()	模式单元符，将一些原子组合成一个大原子使用
+	匹配1个或多次前面的原子		



正则表达式基础知识：量词

➤ 量词

常见的量词包括：*、?、+、{n}、{n,}、{n, m}

#正则表达式 量词

```
content = 'ac abc abbc abbbc abbbbc'
result = re.findall('ab*c', content)
print('*', result)
result = re.findall('ab+c', content)
print('+', result)
result = re.findall('ab{2}c', content)
print('{n}', result)
result = re.findall('ab?c', content)
print('?', result)
```

符号	含义	符号	含义
.	匹配除了换行符以外的任意字符	{n}	前面的原子恰好出现n次
^	匹配字符串的开始位置	{n,}	前面的原子至少出现n次
\$	匹配字符串的结束位置	{n, m}	前面的原子至少出现n次，至多出现m次
*	匹配0次、1次或多次前面的原子		模式选择符，从多个模式中任意选择一个模式匹配
?	匹配0次、1次前面的原子	()	模式单元符，将一些原子组合成一个大原子使用
+	匹配1个或多次前面的原子		



正则表达式基础知识：字符集

➤ 字符集

- 用于匹配一组可能出现的字符。
- 它使用方括号 [] 包围多个字符，表示匹配其中的任意一个。
- 例如：[abc]：匹配 "a"、"b" 或 "c" 中的任何一个字符。
- [0-9]：匹配任意一个数字（等价于 \d）。
- [a-z]：匹配任意一个小写字母。[A-Za-z]：匹配任意一个英文字母（大小写均可）。
- [aeiou]：匹配任意一个元音字母。。

```
content = 'Hello 123 4567 World_This is a Regex Demo, Hello, my phone number is 021-65901447 and email is yuanhy@sufe.edu.cn, and my website is ht  
result = re.findall(r'[021]', content)  
print('result for findall', result)
```



正则表达式基础知识：字符集与括号的区别

➤ 括号 ()

- 在正则表达式中有两种主要用途：
- 分组（Grouping）：将多个字符作为一个整体处理。
- 捕获（Capturing）：用于提取匹配到的子字符串。

```
result = re.findall(r'[021]', content)
print('result for findall', result)

#result = re.findall(r'(021)', content)
#print('result for findall', result)
```



正则表达式基础知识：字符集与括号的区别

➤ 括号 ()

- 在正则表达式中有两种主要用途：
- 分组（Grouping）：将多个字符作为一个整体处理。
- 捕获（Capturing）：用于提取匹配到的子字符串。
- 非捕获组(?:)：剥离提取功能，不需要特定的组的内容

```
result = re.findall(r'[021]', content)
print('result for findall', result)

#result = re.findall(r'(021)', content)
#print('result for findall', result)
```



正则表达式基础知识：贪婪模式与懒惰模式

➤ 贪婪模式与懒惰模式

- 懒惰模式采用就近匹配原则
- 默认情况下是贪婪模式，在贪婪模式的基础上，需要在对应的部分后面加上“？”，就可以转换为懒惰模式。

贪婪模式：如果已经找到了符合条件的字符串，仍然不会停止搜索，直到找到结尾为止；

懒惰模式：一旦搜索到符合条件的字符串，就结束搜索。



正则表达式基础知识：贪婪模式与懒惰模式

```
import re

html = '<p>First <b>bold</b> text</p> <p>Second <i>italic</i> text</p>'
pattern = r"<p>(.*?)</p>"

matches = re.findall(pattern, html)
print(matches)

pattern_lazy = r"<p>(.*?)</p>"

matches_lazy = re.findall(pattern_lazy, html)
print(matches_lazy)
```

['First bold text</p> <p>Second <i>italic</i> text']
['First bold text', 'Second <i>italic</i> text']



正则表达式基础知识：模式修正

➤ 模式修正

- 修正模式可以在不改变正则表达式的情况下，通过模式修正符改变正则表达式的含义，实现一些匹配结果的调整。

```
html = '<P>First <b>bold</b> text</p> <p>Second <i>italic</i> text</p>'

pattern = r"<p>(.*?)</p>"
matches = re.findall(pattern, html)
# matches = re.findall(pattern, html, re.I)
print(matches)

pattern_lazy = r"<p>(.*?)</p>"
matches_lazy = re.findall(pattern_lazy, html)
# matches_lazy = re.findall(pattern_lazy, html, re.I)
print(matches_lazy)
```

符号	含义
I	匹配时忽略大小写
M	多行匹配，多个开头和结尾
U	根据Unicode字符及解析字符，日语或汉字
S	让.匹配包括换行符，即使用了该模式修正后，“.”匹配就可以匹配任意的字符了



正则表达式基础知识：转义字符

- 正则表达式中的**转义字符（Escape Characters）**用于匹配特殊字符或改变字符的默认含义，使其能在正则表达式中被正确解析。
- 在正则表达式中，某些字符具有特殊含义，例如 `.`、`*`、`+`、`?`、`|`、`()`、`[]`、`{}`、`^`、`$`。如果你想匹配它们的原义，需要使用反斜杠 `\` 进行转义。

#正则表达式 转义字符

```
html = '<p>First <b>bold</b> text</p> <p>Second 500$ <i>italic</i> text</p>'
```

```
pattern = r"\d*$"
```

```
#pattern = r"\d*\$"
```

```
matches = re.findall(pattern, html)
```

```
print(matches)
```



正则表达式-例子

例子:

```
import re
```

```
html = "<div id='songs-list'>
<h2 class='title'>经典老歌</h2>
<p class='introduction'>
    经典老歌列表
</p>
<ul id='list' class='list-group'>
    <li data-view='2'>一路上有你</li>
    <li data-view='7'>
        <a href='/2.mp3' singer='任贤齐'>沧海一声笑</a>
    </li>
    <li data-view='4' class='active'>
        <a href='/3.mp3' singer='齐秦'>往事随风</a>
    </li>
    <li data-view='6'><a href='/4.mp3' singer='beyond'>光辉岁月</a></li>
    <li data-view='5'><a href='/5.mp3' singer='陈慧琳'>记事本</a></li>
    <li data-view='5'>
        <a href='/6.mp3' singer='邓丽君'>但愿人长久</a>
    </li>
</ul>
</div>"
```

结果:

一路上有你
沧海一声笑
往事随风
光辉岁月
记事本
但愿人长久

符号	含义
I	匹配时忽略大小写
M	多行匹配
L	做本地化识别匹配
U	根据Unicode字符及解析字符
S	让.匹配包括换行符，即使用了该模式修正后，“.”匹配就可以匹配任意的字符了



推荐正则表达式学习资料

- 韦玮. 精通Python网络爬虫核心技术、框架和项目实战. 机械工业出版社. 2017
- <https://deerchao.cn/tutorials/regex/regex.htm#mission>
- <https://www.runoob.com/python/python-reg-expressions.html>
- 线上正则表达式辅助工具
 - ✓ 开源中国正则表达式测试工具 (<http://tool.oschina.net/regex/>)



课上练习

- 熟悉requests库
- 熟悉正则表达式
- 正则表达式练习

<https://tool.oschina.net/regex/#>

利用正则表达式提取以下网站中的中文电影名及英文电影名

<https://ssr1.scrape.center/>

谢谢!
Thank you

